

ESTRUCTURAS DE DATOS

Ayudantía 6: Árboles

Ignacia Nettle Oliveri
inettle@usm.cl

BLOQUE 1

Problema de Búsqueda

Problema de Búsqueda

Definición

Supongamos que tenemos una colección L de n datos, almacenados de la forma:

$$L = \{(k_1, I_1), (k_2, I_2), \dots, (k_n, I_n)\}$$

Donde I_j es la información asociada a la clave k_j .

Dada una clave particular K , el **problema de búsqueda** consiste en localizar el registro (k_j, I_j) en L , tal que $k_j = K$, si es que existe.

Una **búsqueda exitosa** es aquella en la cual se encuentra un registro en L con clave $k_j = K$. Una **búsqueda no exitosa** es una en la cual **no existe** un registro con clave $k_j = K$.

Nosotros estamos interesados en **búsquedas exactas**, es decir, un registro cuya clave sea exactamente la clave de búsqueda.

TDA Diccionario

Diccionarios

Un diccionario es un TDA que almacena registros **clave-valor** (k, I) donde la clave k es **única** en cada registro. Soporta las siguientes operaciones:

- ▶ Obtener el valor I asociado a la clave k .
- ▶ Insertar un registro (k, I) .
- ▶ Borrar el registro (k, I) .

Tipos de Diccionarios

- ▶ **Diccionarios Ordenados:** sus registros están ordenados en base al **valor de sus claves**.
- ▶ **Diccionarios Desordenados:** sus registros están "desordenados".

TDA Diccionario: Métodos Generales

Método	Descripción
<code>tipoInfo get(tipoClave k)</code>	Obtiene el valor i almacenado en registro (k,i)
<code>void insert(tipoClave k, tipoInfo i)</code>	Almacena el registro (k,i)
<code>tipoInfo remove(tipoClave k)</code>	Elimina el registro (k,i)
<code>void clear()</code>	Reinicia el diccionario, dejándolo vacío
<code>int size()</code>	Retorna la cantidad de registros del diccionario
Métodos Diccionario Ordenado	Descripción
<code>tipoClave* first()</code>	Retorna puntero a la clave de menor valor almacenada.
<code>tipoClav* next(tipoClave ka)</code>	Retorna puntero a la clave kb de menor valor, que sea mayor que ka

TDA Diccionario

Tipos de Implementaciones

Es posible implementar un diccionario mediante **listas** o **arreglos**, pero la búsqueda de registros tomaría $O(n)$ en el peor caso.

Para diccionarios ordenados, la búsqueda de registros puede tardar $O(\log n)$ en el peor caso mediante **búsqueda binaria**, pero **mantener el orden** frente a un borrado es $O(n)$ debido a que hay que reacomodar los registros.

¿Hay una forma más eficiente de hacerlo?

- ▷ **Métodos basados en árboles:** complejidad típica de $O(\log n)$ en el peor caso.
- ▷ **Métodos basados en hashing:** complejidad promedio de $O(1)$.

BLOQUE 2

Árboles Generales

Árboles Generales

Definición

Los árboles son estructuras que permiten tanto **acceso y actualización eficiente** de sus elementos.

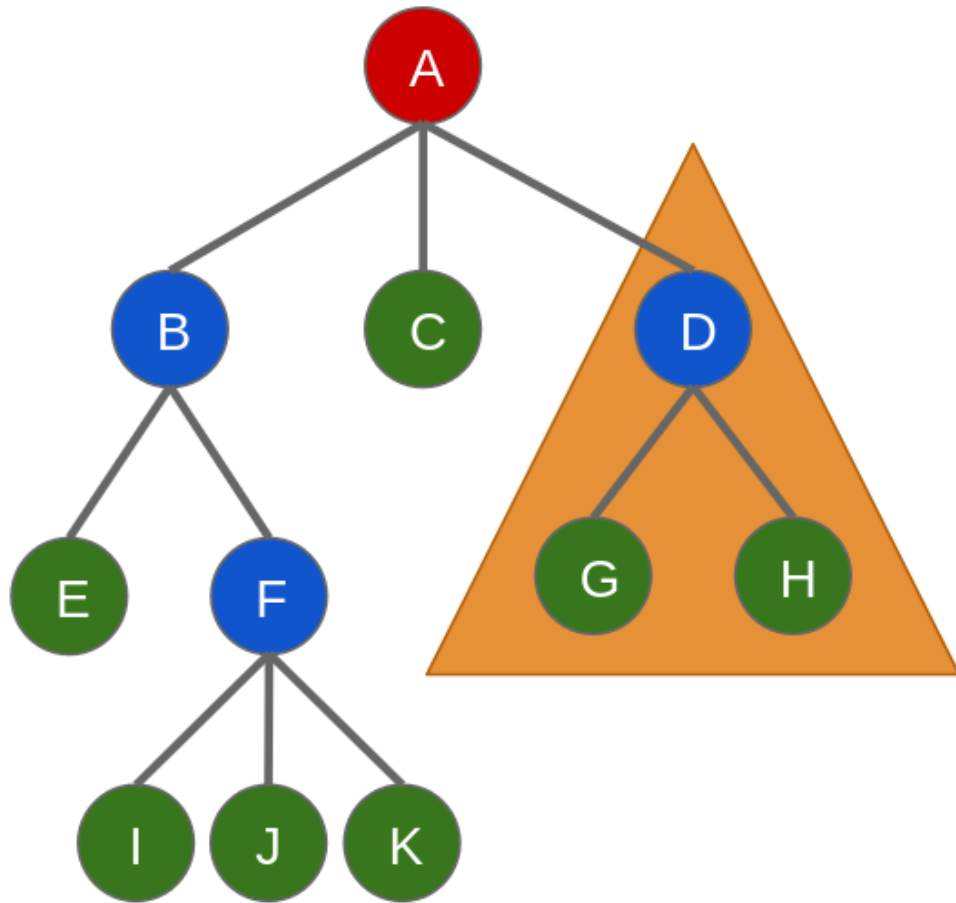
Sin embargo, a diferencia de las estructuras estudiadas hasta ahora, en los árboles **no existe secuencialidad entre elementos**, sino una estructura jerárquica, de manera que un elemento tiene un solo predecesor, pero **puede tener varios sucesores**.

Componentes

Un árbol es una colección de elementos llamados **nodos** que tienen una relación *padre-hijo* entre sí. Los nodos se clasifican de acuerdo a la posición que ocupan dentro del árbol:

- ▷ **Raíz:** único nodo sin padre.
- ▷ **Nodo interno/interior:** tiene un padre y al menos un hijo.
- ▷ **Nodo hoja:** tiene un padre y sin hijos.
- ▷ **Subárbol:** árbol formado por un nodo y sus descendientes, dentro del árbol principal.

Árboles Generales



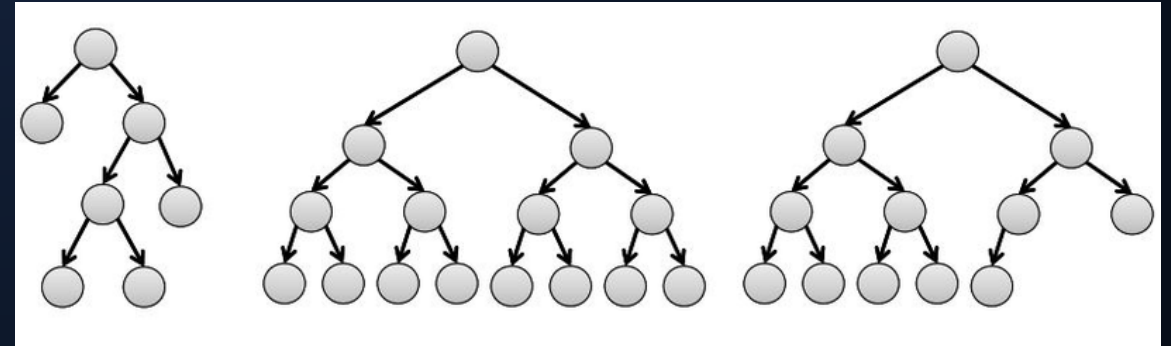
Conceptos Importantes

- ▷ **Grado de un Nodo:** número de hijos o subárboles de un nodo.
- ▷ **Grado del árbol:** número máximo de hijos que tienen los nodos de un árbol
- ▷ **Profundidad:** número de predecesores de un nodo, o largo del único camino desde la raíz al nodo.
- ▷ **Altura:** profundidad máxima de los nodos.
- ▷ **Nivel:** número de nodos del camino desde la raíz hasta el nodo. Raíz se considera nivel cero.

Árboles Generales

Conceptos Importantes

- ▷ **Árbol completo:** si todo nodo interno tiene exactamente el grado del árbol.
- ▷ **Árbol lleno:** si es completo y todas sus hojas están en el mismo nivel.
- ▷ **Árbol casi-lleno:** árbol lleno hasta el penúltimo nivel. Hojas en el último nivel están lo más a la izquierda posible.



BLOQUE 3

Recorrido de Árboles

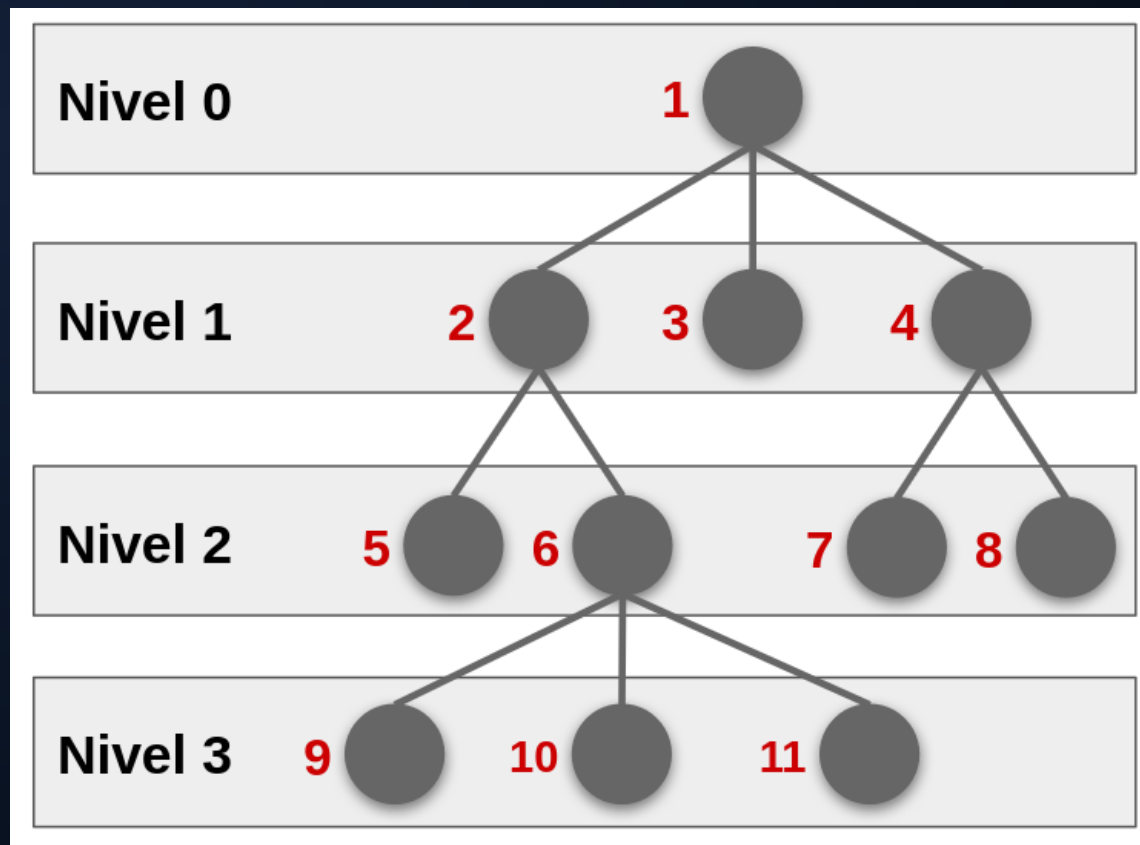
Recorrido en anchura, en profundidad, pre-orden, in-orden y post-orden

Recorrido en anchura (BFS)

Definición

En este tipo de recorrido, el árbol se visita **por niveles**, es decir, se visitan todos los nodos de un nivel antes de pasar al siguiente.

Para implementarlo, se usa una **cola**: primero se visita la raíz, y se colocan todos sus hijos en la cola. Luego, por cada nodo en la cola, se procede de la misma manera.



Recorrido en profundidad (DFS)

Definición

En este tipo de recorrido, cada paso implica moverse hacia un nodo de **mayor profundidad** hasta alcanzar una hoja.

Este recorrido se puede llevar a cabo de tres formas distintas. Para cada recorrido, diremos que tenemos tres acciones posibles:

- ▷ **Left:** ir lo más profundo posible en el subárbol izquierdo.
- ▷ **Right:** ir lo más profundo posible en el subárbol derecho.
- ▷ **Root:** visitar y procesar el nodo actual.

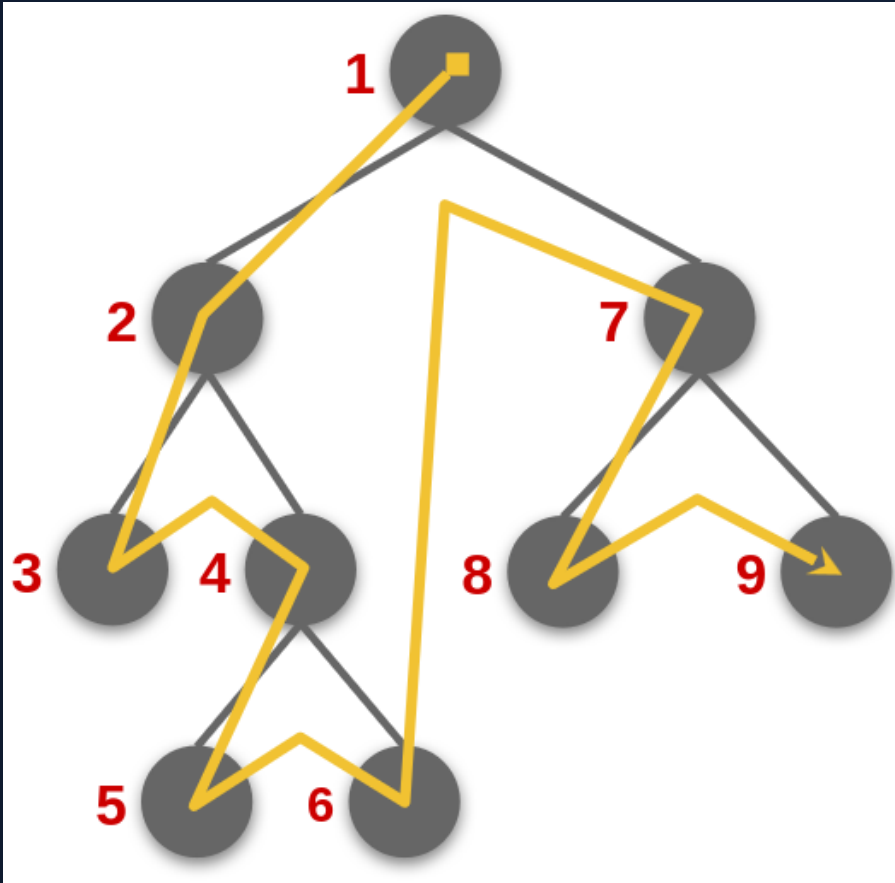
Tipos de Recorridos de DFS

Considerando las tres acciones posibles mencionadas anteriormente, tenemos los siguientes tipos de recorridos para el DFS:

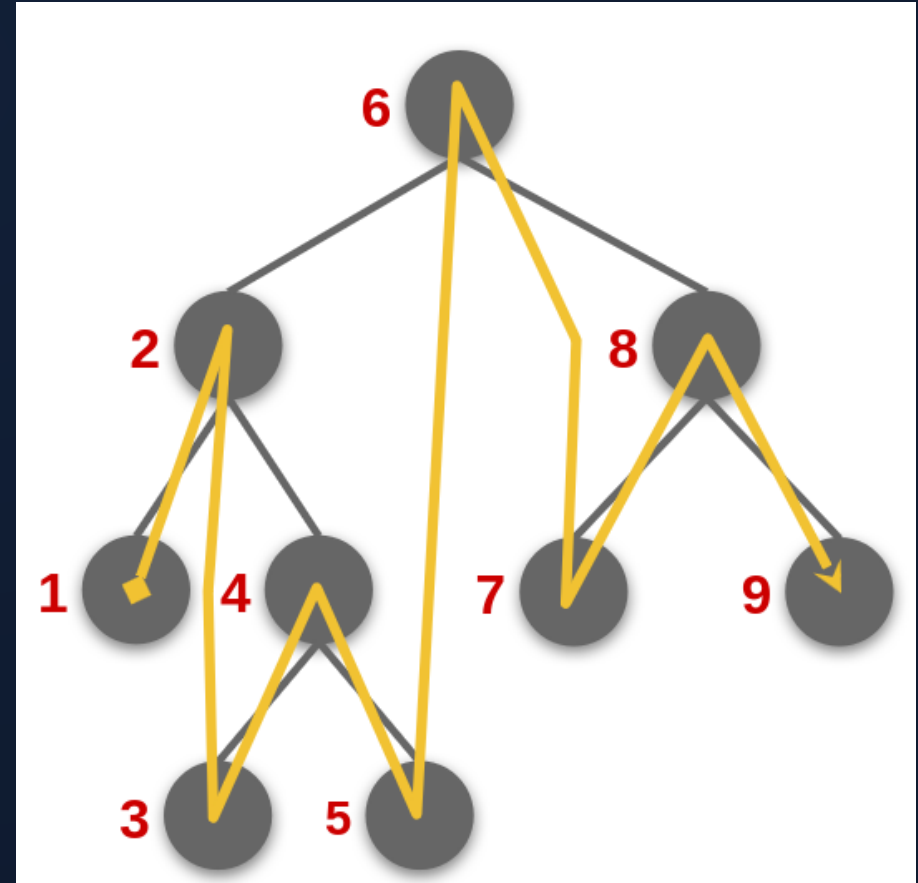
- ▷ **Pre-Orden:** Root → Left → Right
- ▷ **In-Orden:** Left → Root → Right
- ▷ **Post-Orden:** Left → Right → Root

Recorrido en Profundidad: Ejemplo

Pre-orden

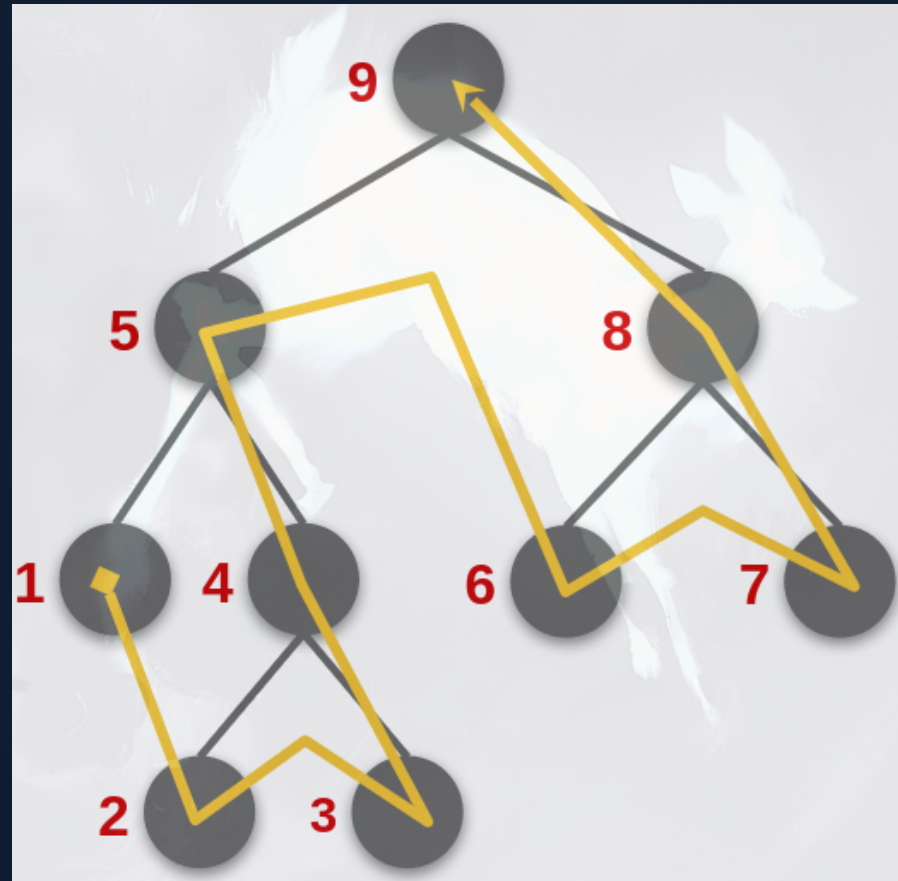


In-Orden



Recorrido en Profundidad: Ejemplo

Post-Orden



Recorrido en Profundidad: Código

```
void pre_orden(tNodo *nodo){
    if(nodo == nullptr) return;

    cout << nodo->info << " ";
    pre_orden(nodo->izq);
    pre_orden(nodo->der);
}

void in_orden(tNodo *nodo){
    if(nodo == nullptr) return;

    in_orden(nodo->izq);
    cout << nodo->info << " ";
    in_orden(nodo->der);
}
```

```
void post_orden(tNodo *nodo){
    if(nodo == nullptr) return;

    post_orden(nodo->izq);
    post_orden(nodo->der);
    cout << nodo->info << " ";
}
```

BLOQUE 4

Árboles Binarios de Búsqueda (ABB)

Árboles Binarios de Búsqueda (ABB)

Definición

Se define como un árbol binario (árbol de grado 2) que almacena elementos de un conjunto, y permite buscar por ellos.

Para una búsqueda eficiente el árbol **debe** estar ordenado de tal forma que el hijo izquierdo tenga un valor menor que el del padre, y el hijo derecho un valor mayor.

```
bool find(tNodo *nodo, tElem item){  
    if(nodo == NULL) return 0; //item no está en ABB  
    if(nodo->info == item) return 1; //se encontró item  
    if(item < nodo->info)  
        return(find(nodo->izq, item));  
    else  
        return find(nodo->der, item);  
}
```

Árboles Binarios de Búsqueda (ABB)

Inserción

Para insertar un elemento en un ABB se debe ir comparando el valor del elemento con los nodos del árbol, empezando por la raíz, hasta llegar a una hoja y luego insertarlo como hijo de este.

Inserción **sólo ocurre en hojas**.

Borrado

A diferencia de la inserción, es posible eliminar nodos **en cualquier punto del árbol**, pero es necesario mantener el orden del ABB. Existen 3 casos de borrado:

- ▷ **Borrado de una hoja:** simplemente se borra el nodo.
- ▷ **Borrado de un nodo con un hijo:** nodo a borrar se reemplaza por el hijo.
- ▷ **Borrado de un nodo con dos hijos:** nodo a borrar se reemplaza por su **sucesor** o **predecesor** (en valor) y luego recursivamente se elimina el nodo repetido conforme a los casos anteriores.

BLOQUE 5

Ejercicios

Ejercicio 1

Construye un árbol binario a partir de la siguiente secuencia de números:

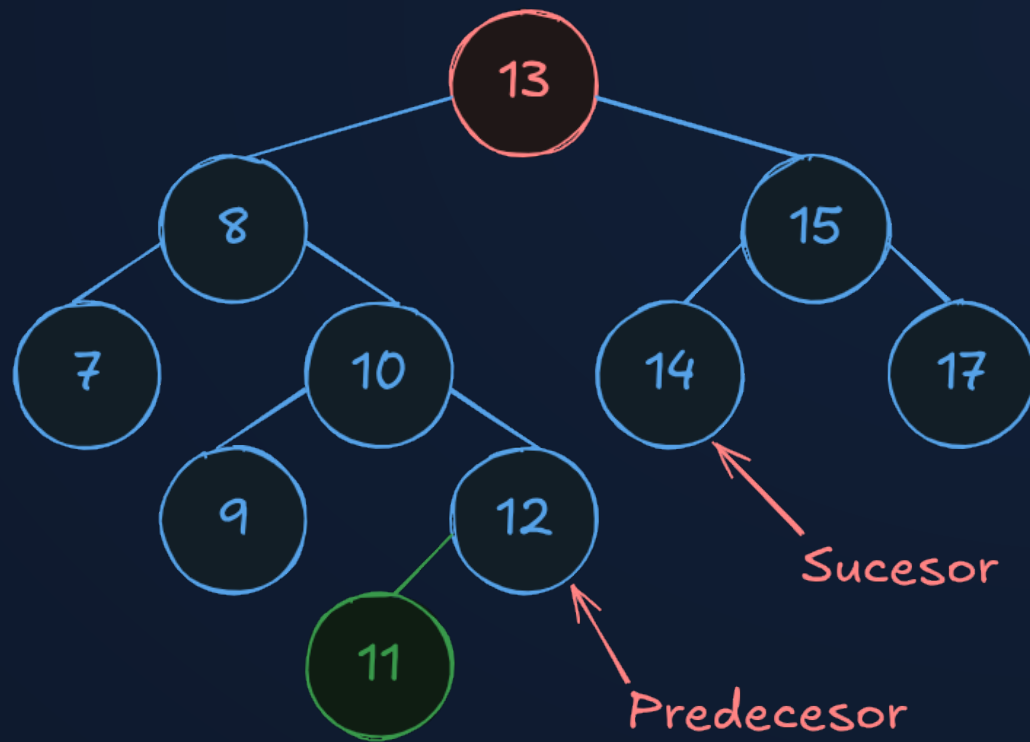
$\{13, 15, 8, 10, 9, 14, 17, 7, 12\}$,

Luego:

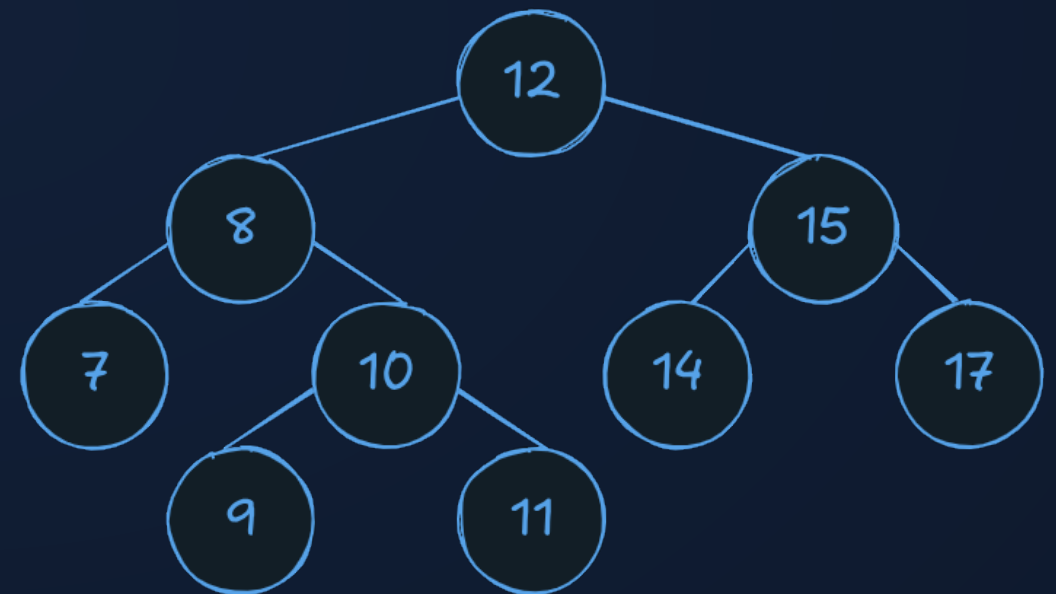
1. Insertar el **11**.
2. Eliminar el **13**.

Ejercicio 1 - Solución

Post-Inserción



Post-Borrado



Ejercicio 2

A partir del árbol binario obtenido en el ejercicio anterior, muestra el orden en que se visitan los nodos si se recorre el árbol en:

- ▷ Pre-orden
- ▷ In-orden
- ▷ Post-orden

Ejercicio 2 - Solución

- ▷ Pre-orden: 12, 8, 7, 10, 9, 11, 15, 14, 17
- ▷ In-orden: 7, 8, 9, 10, 11, 12, 14, 15, 17
- ▷ Post-orden: 7, 9, 11, 10, 8, 14, 17, 15, 12

Si se dan cuenta, en el recorrido pre-orden la raíz siempre está al principio de la secuencia, en in-orden al medio y en post-orden al final.

Ejercicio 3

A partir del árbol binario obtenido en el ejercicio 1, elimina cada nodo siguiendo la siguiente secuencia de borrado:

$\{8, 15, 10, 7, 9, 14, 12, 17, 11\}$

La estructura del ABB se debe **matener en cada paso**.

CIERRE

¡Eso es todo 🧐!